

DBSCAN and Agglomerative Clustering

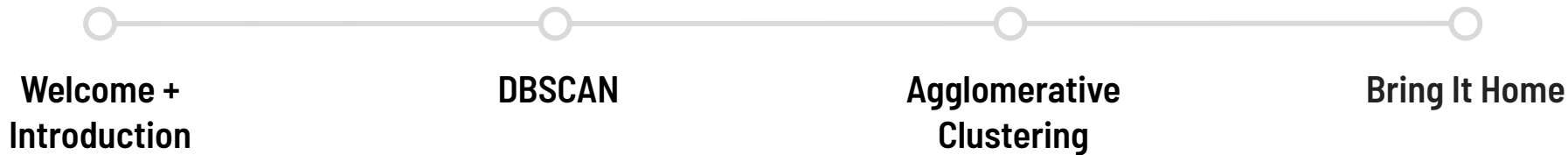
Unsupervised Learning

Advanced Modeling



GENERAL ASSEMBLY

Lesson roadmap



Learning objectives



1

Describe the effect of **epsilon** and **min_points** on DBSCAN.

2

Identify advantages and disadvantages of DBSCAN.

3

Implement DBSCAN using **scikit-learn**.

4

Describe the agglomerative clustering technique, including advantages and disadvantages.

5

Implement agglomerative clustering using **scikit-learn**.

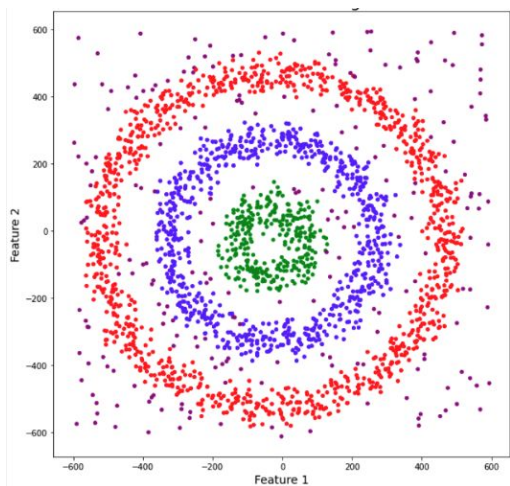
A black and white photograph of a train station platform. Several people are walking away from the camera towards a bright light source at the end of the platform, creating long silhouettes. The floor is wet and reflective. A red vertical bar is on the left side of the image.

Warm Up

Clusters that break the rules

Clustering

- ☐ Would K-Means work here? Why or Why not?
- ☐ What could go wrong with centroid-based clustering



Review K-Means

Unsupervised Learning

K-Means Clustering

In unsupervised learning, one strategy is to **cluster** observations into **groups**.

Observations of the same group are more similar than observations in different groups.

We've learned one clustering algorithm so far: **k-means clustering**.



Trivia

What are the pros & cons of K-Means?

DBSCAN

Unsupervised Learning

DBSCAN

There's another method of clustering that can sidestep some of the disadvantages of k-means: **DBSCAN**

DBSCAN stands for **D**ensity-**B**ased **S**patial **C**lustering of **A**pplications with **N**oise

DBSCAN

There's another method of clustering that can sidestep some of the disadvantages of k-means: **DBSCAN**

DBSCAN stands for **D**ensity-**B**ased **S**patial **C**lustering of **A**pplications with **N**oise.

We detect areas of high density and low density:

- ☐ We cluster in areas of high density
- ☐ We avoid clustering in low-density areas

Hyperparameters

DBSCAN requires you to specify two hyperparameters:

1. **min_samples**: the minimum number of points needed to form a cluster
2. **epsilon**: the “searching” distance when attempting to build a cluster



Hyperparameters

DBSCAN requires you to specify two hyperparameters:

1. **min_samples**: the minimum number of points needed to form a cluster
2. **epsilon**: the “searching” distance when attempting to build a cluster

THIS IS A HUGE DISADVANTAGE OF DBSCAN!

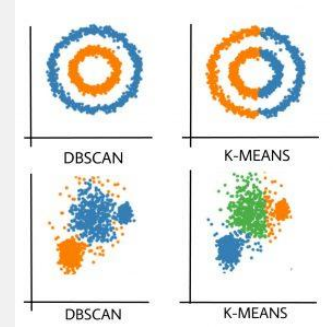
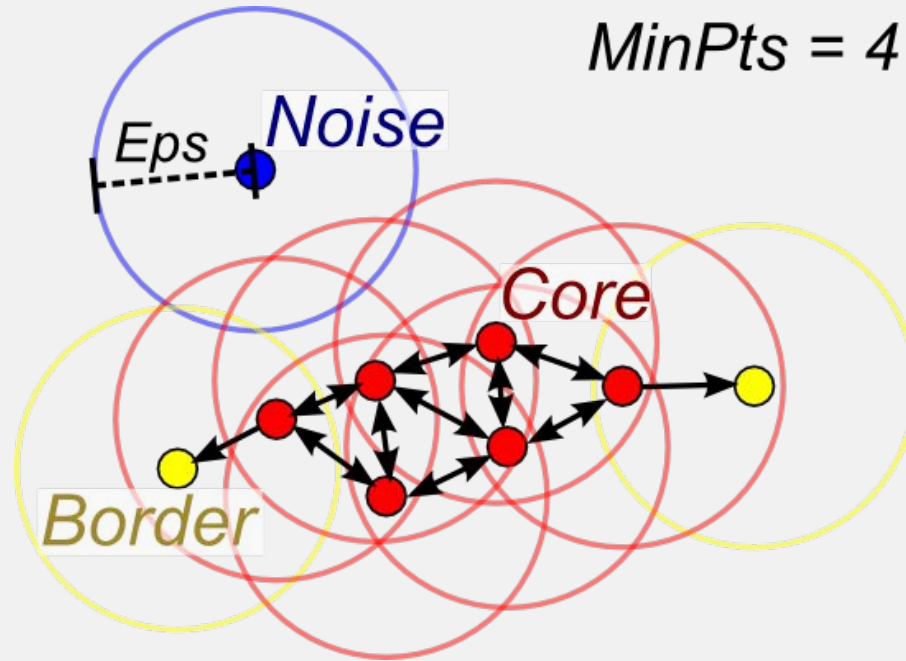


Visualizing DBSCAN

A picture is worth a thousand algorithms:

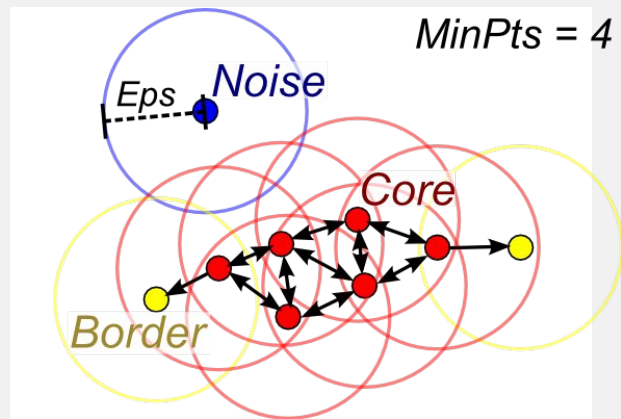
<https://www.naftaliharris.com/blog/visualizing-dbscan-clustering/>

Visualizing DBSCAN



Visualizing DBSCAN

- **Core points:** Points inside a cluster that have at least **min_samples** points within **epsilon**.
- **Border points:** Points inside a cluster that do not have at least **min_samples** points within **epsilon**
- **Noise:** Points that belong to no cluster



Why DBSCAN?

DBSCAN allows us to detect some cluster patterns that k-means might not be able to detect. k-means wants sphere-ish blobs, where **DBSCAN wants densely connected blobs of any contiguous shape.**

We don't need to pre-specify the number of clusters; **DBSCAN will determine how many clusters are appropriate given fixed `min_samples` and `epsilon` values.** This is particularly valuable when we are clustering data in more than two or three dimensions.

Not every point is clustered! Good for identifying outliers.



Let me say that again: DBSCAN Pros and Cons

Pro	Con	k-means...
It handles blobs of any contiguous shape	... but we need the data to have approximately uniform density throughout	Can only handle spheroid blobs
It tells us how many clusters there are!	... assuming we picked its two (2!) tuning parameters appropriately	We have to select k, but at least there's only one tuning parameter!
It finds outliers for us!	... but if we picked our tuning parameters poorly, it will look like there are too many outliers	Enforces everything to be in <i>some</i> cluster, even if it doesn't make sense!

DBSCAN

Let's see it for ourselves!

- Open **02-dbscan-agglom-clustering.ipynb**



Agglomerative Clustering

Unsupervised Learning

Hierarchical Clustering

Another class of clustering algorithms is called **hierarchical clustering**, and they can be divided into two subcategories:

- **Agglomerative clustering**: Start with every observation in its own cluster, and merge them into bigger clusters one-by-one until some desired number of clusters is achieved.
- **Divisive clustering**: Start with every observation in one big cluster, and slowly split clusters apart until some desired number of clusters is achieved

Hierarchical Clustering

Another class of clustering algorithms is called **hierarchical clustering**, and they can be divided into two subcategories:

- **Agglomerative clustering:** Start with every observation in its own cluster, and merge them into bigger clusters one-by-one until some desired number of clusters is achieved.
- *Divisive clustering: Start with every observation in one big cluster, and slowly split clusters apart until some desired number of clusters is achieved*

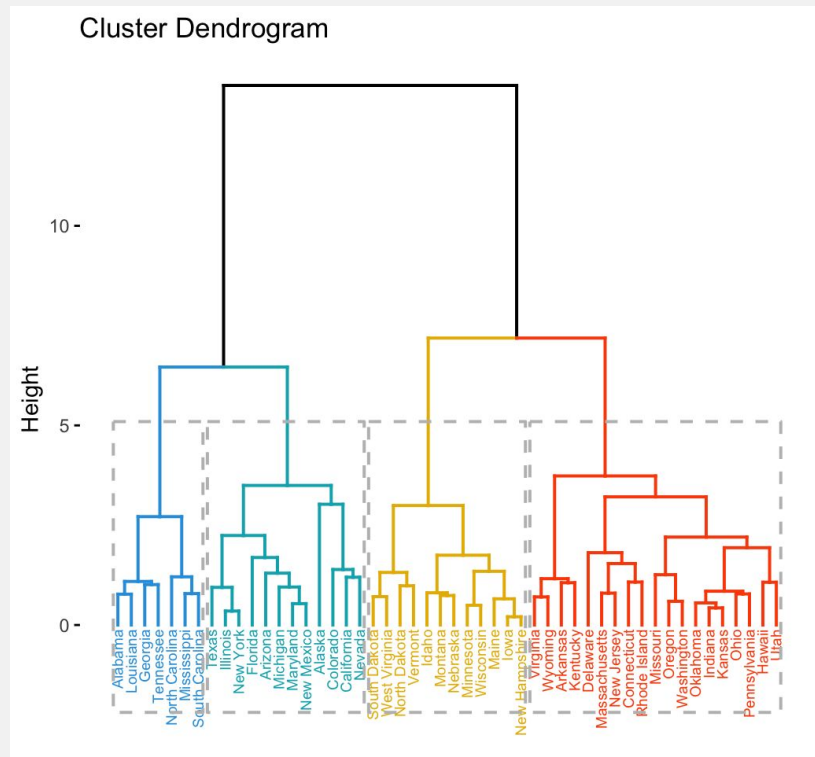
Agglomerative Clustering

Between the two, almost always **agglomerative clustering** is preferred. In **scikit-learn**, you get to tune two relevant arguments:

- **n_clusters**: The desired number of clusters in your result
- **linkage**: The metric used to merge clusters
 - **ward** (default): Minimizes the variance of clusters being merged
 - **single** (popular): Uses the minimum distances between each point and each cluster. Using this option results in a special algorithm called **single-linkage clustering**.



Agglomerative Clustering Dendrogram



Agglomerative clustering involves starting at the bottom and merging clusters until All is One.

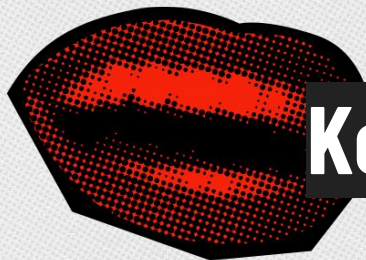
Agglomerative Clustering

Let's see it for ourselves!

- Open **02-dbscan-agglom-clustering.ipynb**



Bring It Home



Key takeaways



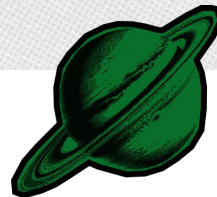
DBSCAN clusters based on density

Useful alternative to distance-metric models, but you need to specify 2 tuning parameters



Agglomerative clustering is hierarchical

Starts with all observations as individual clusters, then aggregates



There are many clustering models to explore

scikit-learn has many, and k-medoids clustering is another good model

